

Tutorial: Hyperparameter Tuning with Logistic Regression in scikit-learn

- Grid Search (GridSearchCV)
- Random Search (RandomizedSearchCV)

1. Setup: Classification Task

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline

# Load dataset
X, y = load_breast_cancer(return_X_y=True)

# Split the dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Define pipeline
pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('logreg', LogisticRegression(solver='liblinear'))
    # solver chosen for compatibility with small datasets
])
```

2. What are Hyperparameters in Logistic Regression?

Hyperparameter	Description
c	Inverse of regularization strength (smaller = stronger regularization)
penalty	Type of regularization ('l1' , 'l2')
solver	Optimization algorithm ('liblinear' supports 'l1' and 'l2')

3. Grid Search (GridSearchCV)

Try all combinations of hyperparameter values.

```
from sklearn.model_selection import GridSearchCV
# Define grid
param_grid = {
    'logreg__C': [0.01, 0.1, 1, 10, 100],
    'logreg__penalty': ['l1', 'l2']
}
grid = GridSearchCV(pipe, param_grid, cv=5, scoring='accuracy', n_jobs=-1)
grid.fit(X_train, y_train)

print("Best Parameters:", grid.best_params_)
print("Best CV Accuracy:", grid.best_score_)
```

4. Random Search (RandomizedSearchCV)

More efficient for large hyperparameter spaces.

```
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import loguniform

# Define distributions
param_dist = {
    'logreg__C': loguniform(0.001, 100), # continuous distribution
    'logreg__penalty': ['l1', 'l2']
}

random_search = RandomizedSearchCV(
    pipe,
    param_distributions=param_dist,
    n_iter=20,
    cv=5,
    scoring='accuracy',
    random_state=42,
    n_jobs=-1
)
random_search.fit(X_train, y_train)

print("Best Parameters (Random Search):", random_search.best_params_)
print("Best CV Accuracy (Random Search):", random_search.best_score_)
```

5. Evaluate the Best Model on Test Set

```
from sklearn.metrics import classification_report

# Get best model from grid search
best_model = grid.best_estimator_
y_pred = best_model.predict(X_test)

print(classification_report(y_test, y_pred))
```

6. Tips and Best Practices

Tip	Description
Use Pipelines	Always include preprocessing like scaling
Prefer RandomizedSearchCV first	More efficient, especially for large search spaces
Use cross-validation	Avoids overfitting to a single split
Monitor training/test performance	To detect overfitting or underfitting
Avoid data leakage	Don't scale or preprocess outside the pipeline

Challenge: Try It Yourself

Apply hyperparameter tuning to:

- LogisticRegressionCV (built-in CV version)
- RidgeClassifier for comparison
- A dataset like `digits` or your own!